

1 Git :

1.1 如果要避免不需要提交的檔案，出現於 unstaged change 欄位：

首先，下載.gitignore 檔。

<https://github.com/github/gitignore/blob/main/Global/Eclipse.gitignore>

接著執行以下指令：

命令提示字元

```
C:\Users\cuser\git\TestRestBookingProject>type nul > .gitignore
```

執行該指令後，便會於專案根目錄，產生.gitignore 檔，這時就將剛剛下載下來的.gitignore 檔，其內容通通貼到這個新產生的.gitignore 檔，並存檔。

```
C:\Users\cuser\git\TestRestBookingProject>dir /a
磁碟區 C 中的磁碟沒有標籤。
磁碟區序號: 1232-4387

C:\Users\cuser\git\TestRestBookingProject 的目錄
2025/09/13 下午 08:32 <DIR> .
2025/09/13 下午 08:32 <DIR> ..
2025/09/13 下午 08:26 <DIR> .git
2025/09/13 下午 08:32 1,008 .gitignore
2025/09/13 下午 08:31 948 .gitignore.gitignore
2025/09/13 下午 07:59 <DIR> .metadata
2024/08/27 下午 10:09 <DIR> RestBookingProject
2 個檔案 1,956 位元組
5 個目錄 846,772,740,096 位元組可用

C:\Users\cuser\git\TestRestBookingProject>git rm -r --cached .metadata
fatal: pathspec '.metadata' did not match any files

C:\Users\cuser\git\TestRestBookingProject>git commit -m "stop tracking .metadata"
On branch master
Your branch is up to date with 'origin/master'.

Changes not staged for commit:
  (use "git add/rm <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        deleted:    .gitignore.gitignore

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        .gitignore

no changes added to commit (use "git add" and/or "git commit -a")
```

最後回到 egit 視圖，重新整理即可。

1.2 如果要讓檔案，完全回復到遠端版本：

右鍵檔案->Replace With->HEAD Revision(恢復成本地分支最後一次 commit 的內容)。

1.3 如何將多餘的檔案(.settings 跟 target 內檔案)移出 git 的管理，以讓 egit 的 unstaged、staged change 都沒有這些檔案：

1.3.1 首先先檢查要移除的檔案：

```
C:\Users\cuser\git\TestRestBookingProject>git ls-files | findstr target
```

1.3.2 接著移除：

```
C:\Users\cuser\git\TestRestBookingProject>git rm -r --cached  
RestBookingProject/target
```

```
C:\Users\cuser\git\TestRestBookingProject>git rm -r --cached  
RestBookingProject/.settings
```

1.3.3 再次檢查檔案是否被移除：

```
C:\Users\cuser\git\TestRestBookingProject>git ls-files | findstr target
```

```
C:\Users\cuser\git\TestRestBookingProject>git ls-files | findstr settings
```

1.3.4 接著就可正式將這些檔案，移出 git 管理：

```
git commit -m "remove .settings and target from version control"
```

1.3.5 檢查結果是否是預期的：

```
C:\Users\cuser\git\TestRestBookingProject>git status
```

如果 Changes not staged for commit、Untracked files 都沒有 .settings 與 target，即為預期結果。

1.3.6 檔案又退回到 staged change，所有檔案圖示，都有

一個灰色叉叉：

因為尚未執行

```
C:\Users\cuser\git\TestRestBookingProject>
```

```
git commit -m "remove .settings and target from version control"
```

2 如何啟動 HTTPS：

透過 Spring Boot 的 application.properties：

2.1 application.properties：

於 application.properties，啟動內嵌 Tomcat 的 SSL、設置 https 憑證，且伺服器端以 8443 為埠號。

```
server.ssl.enabled=true
```

```
server.ssl.key-store=classpath:keystore.p12
```

```
server.ssl.key-store-password=606452441716
```

```
server.ssl.key-store-type=PKCS12
```

```
server.port=8443
```

2.2 Spring Boot 跟 Spring MVC 專案的 classpath？

classpath 可以直接當成 src/main/resources；在開發習慣上可以這樣想，但實際上不完全等於。

2.3 server.ssl.key-store=classpath:keystore.p12

寫成這樣，為何路徑不會被解析成：

```
src/main/resourceskeystore.p12
```

而可以是 src/main/resources/keystore.p12？

classpath:keystore.p12，指的是「class path 根目錄下的 keystore.p12」，

而非直接去拼接 `src/main/resources/keystore.p12`。

2.4 如何查看自簽憑證：

```
keytool -list -v -storetype PKCS12 -keystore
```

```
C:\Users\cuser\git\TestRestBookingProject\RestBookingProject\src\main\resources\keystore.p12
```

3 註冊功能：

3.1 註冊頁面：

`reqAndLogin5.jsp`



針對 email 欄位與密碼欄位，分別檢查 email 格式與兩個密碼是否一致：

3.1.1 檢查 email 格式：

於 email 欄位設置 `type="email"`；瀏覽器會檢查 email 基本格式，若沒通過檢查，瀏覽器會自動加上 `invalid` 偽類。

這時再給該欄位所在區塊，添加 `was-validated` 這個 class；`was-validated` 是 bootstrap 的 class，讓合法的(`:valid`)input 欄位框變成綠色，不合法的(`:invalid`)input 欄位框變成紅色，且 `invalid-feedback` 的 display 變成

block(invalid-feedback 本來是 display: none;)。

該欄位所在區塊：

```
var emailDiv = document.getElementsByClassName("emailDiv");
```

添加 was-validated 這個 class：

```
emailDiv[0].classList.add("was-validated");
```

若擔心該類會重複被添加，那就在 add 之前先添加

```
emailDiv[0].classList.remove('was-validated');
```

而該檢查是 keyup 事件發生當下進行的，故將其包在 keyup 事件內：

```
$("#reg #email").keyup(function(){  
  
    emailDiv[0].classList.remove('was-validated');  
  
    emailDiv[0].classList.add("was-validated");  
});
```

畫面：



請輸入Email：

some999816@gmail

請輸入有效的 Email

3.1.2 檢查兩個密碼是否一致：

取出這兩個欄位值進行檢查，若不一致就對表單內，該元素的 setCustomValidity 寫入值，例如空白字元，這樣該元素自動被瀏覽器添加:invalid 偽類。這時一樣給該欄位所在區塊，添加 was-validated 這個 class。

密碼欄位：

```
var checkPassword = document.getElementById("checkedPassword");
```

該欄位所在區塊：

```
var pwdDiv = document.getElementsByClassName("pwdDiv");
```

對表單內，該元素的 setCustomValidity 寫入值，例如空白字元：

```
checkPassword.setCustomValidity(" ");
```

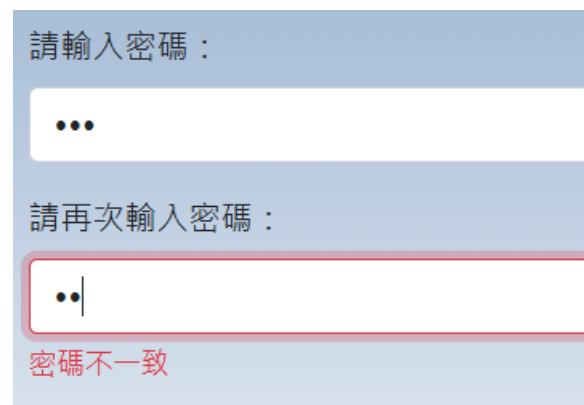
給該欄位所在區塊，添加 was-validated 這個 class：

```
pwdDiv[0].classList.add("was-validated");
```

而這是在該欄位發生 input 事件時要做的檢查：

```
checkPassword.addEventListener("input",function(e){  
    checkPwdMatch();  
});
```

畫面：



3.1.3 傳送註冊表單：

使用 JQuery AJAX，這樣才能在回呼函數，呈顯註冊成功或失敗的視窗。

```
document.getElementById('regSubmitButton').addEventListener('click',function(e){
    var regForm = document.getElementById('reg');
    if(regForm.checkValidity()){
        var regFormData = new FormData(regForm);

        $.ajax({
            url:
                "${pageContext.request.contextPath}/entry/regForUser",
            type: "post",
            data: regFormData,
            processData: false,
            contentType: false,
            success: function(response) {
                if(response=="用戶驗證信已寄出"){
                    $('#regSuccess.modal').modal('show');
                }
            },
            error: function(xhr, status, error) {

            }
        });
    }
})
```

checkValidity()是瀏覽器提供的、驗證表單的 API，可直接使用。

使用 `FormData` 物件是為了最好的包含表單內所有類型的資料；如果用 JQuery 的 `serialize()` 就無法包含圖片。

那兩個 `false` 是為了處理傳送圖片而設置。

JQuery ajax 回呼函數參數，為 `response body` 的內容，可直接用或用 `JSON.parse()` 處理；`fetch()` 則回一個 `Response` 物件，你得再呼叫 `res.text()` 或 `res.json()` 才拿得到 `response body` 的內容。

```
else{
    e.preventDefault();
    e.stopPropagation();

    regForm.classList.add("was-validated");
}
```

若表單驗證失敗，則停止事件預設行為，並阻止事件上浮到上層元素而被攔截，並給欄位所在區塊添加 `was-validated` 類。

畫面：

登入或註冊

註冊 登入 忘記密碼?

● 一般客戶端 ● 餐廳端

請輸入帳號：
109

請輸入Email：
some999816@gmail.com ✓

請輸入密碼：

請再次輸入密碼：
*** ✓

個人/餐廳圖像：
選擇檔案 未選擇任何檔案

提交 取消

3.2 後端：

後端要儲存註冊資料，但註冊的用戶此時尚未啟動，需要產生含驗證碼 `token` 的驗證連結，透過用戶給的 `email` 傳給用戶，用戶須點擊該連結以啟動註冊用戶。

`user` 跟驗證碼 `token`，因為一 `user` 可以有多個驗證信，故其是一對多的關係，

所以驗證碼 token 必然是獨立的 bean；故設置驗證碼 bean，以及對應的表格。

新增表格需加上

ENGINE=InnoDB

DEFAULT CHARACTER SET utf8mb4

COLLATE utf8mb4_0900_ai_ci;

3.2.1 驗證碼 bean：

VerificationToken：

@Column

private boolean used;

因為 token 是一次性的，故以該屬性表示是否已被用過；預設為 false。

最重要的是其跟 user bean 的綁定，其有 user 外鍵，其又跟 user 為多對一關係故用 @ManyToOne、@JoinColumn：

@ManyToOne

@JoinColumn(name="user_account")

private User user;

又有外鍵的那一方，可透過外鍵去主控另一方內容，故這時驗證碼 bean 是主控方。

而在 user bean 也要做出修改，因為其跟驗證碼是一對多的關係，故用 @OneToMany；這時 user bean 是被控方，要標示被主控方的哪個部分主控，就是主控方的外鍵，也就是主控方外鍵對應的屬性，故要寫成：

@OneToMany(mappedBy="user")

private List<VerificationToken> verificationToken;

user 是雙向關係的被控端，那 mappedBy 表示被誰主控，主控方是 Token，因為其有 user 這個對應外鍵的屬性，故屬性 user 能主控所有關連到的 user。故 mappedBy 為 user。

3.2.2 Controller：

EntryController：

儲存註冊資料：

@PostMapping(value="/regForUser")


```
@ResponseBody
public String registerToDB(@Valid RegisterDTO registerDTO){
```

```
    String token = userService.register(registerDTO);
```

若表單欄位名直接等於 DTO 屬性名，則 Spring MVC 可直接將表單值對應到 DTO 屬性值。

@Valid 是對表單值啟用後端驗證，可不加。

3.2.3 Service :

UserService :

3.2.3.1 設置 encoder :

```
PasswordEncoder encoder;
```

需要 encoder，因為等等要把密碼從明文變成密文。

這名稱要跟

```
@Bean public PasswordEncoder passwordEncoder() { return new
BCryptPasswordEncoder(); }
```

搭配？不用，型別有對上即可。

```
public UserService(UserDao userDao, PasswordEncoder encoder,
VerificationRepository verificationRepository) {
    this.encoder = encoder;
}
```

透過建構子注入，注入註冊過的 encoder。

3.2.3.2 儲存用戶 :

```
@Transactional("jpaTxManager")
public String register(RegisterDTO reg) {
    u.setPassword(encoder.encode(reg.getPassword()));

    u.setEnabled(false);
    int result = userDao.addUser(u);
```

存入 dao 前，將密碼從明文變成密文。

用戶狀態設為尚未啟動。

呼叫 `dao` 方法，該方法由 `Spring JDBC` 寫成。

既然該方法由 `Spring JDBC` 寫成，那麼為何交易管理器用 `Spring Data JPA` 的？因為後面要呼叫的 `DAO` 方法由 `Spring Data JPA` 寫成，而 `Spring Data JPA` 版交易管理器，相容 `Spring JDBC`。

若在 `Service` 內有分別用 `Spring JDBC`、`Spring Data JPA` 的 `DAO` 方法，那直接用 `@Transactional` 可以根據實際狀況，切換不同的交易管理器？

不行。

3.2.3.3 產生驗證碼：

需要分別產生明文版跟密文版：

明文版，要附加在驗證連結中：

```
byte[] key = KeyGenerators.secureRandom(32).generateKey();
String tokenForEmail =
Base64.getUrlEncoder().withoutPadding().encodeToString(key);
```

密文版，要寫入資料庫：

```
md = MessageDigest.getInstance("SHA-256");

String tokenHash =
java.util.HexFormat.of().formatHex(md.digest(tokenForEmail.getBytes(StandardCharsets.UTF_8)));

VerificationToken vt = new VerificationToken();
    vt.setToken(tokenHash);
    vt.setUser(u);
    vt.setCreatedAt(Instant.now());

verificationRepository.save(vt);
```

驗證碼生成時間，選擇 `Instant.now()`，是因為其最能鎖定在某個時間。

3.2.3.4 回傳明文版驗證碼：

```
return tokenForEmail;
```

3.2.4 Spring Data JPA Repository :

```
public interface VerificationRepository extends
JpaRepository<VerificationToken, Integer> {

}
```

在 Spring Data JPA , tokenRepo extends JpaRepository 就可呼叫 save() 。

3.2.5 回到 Controller :

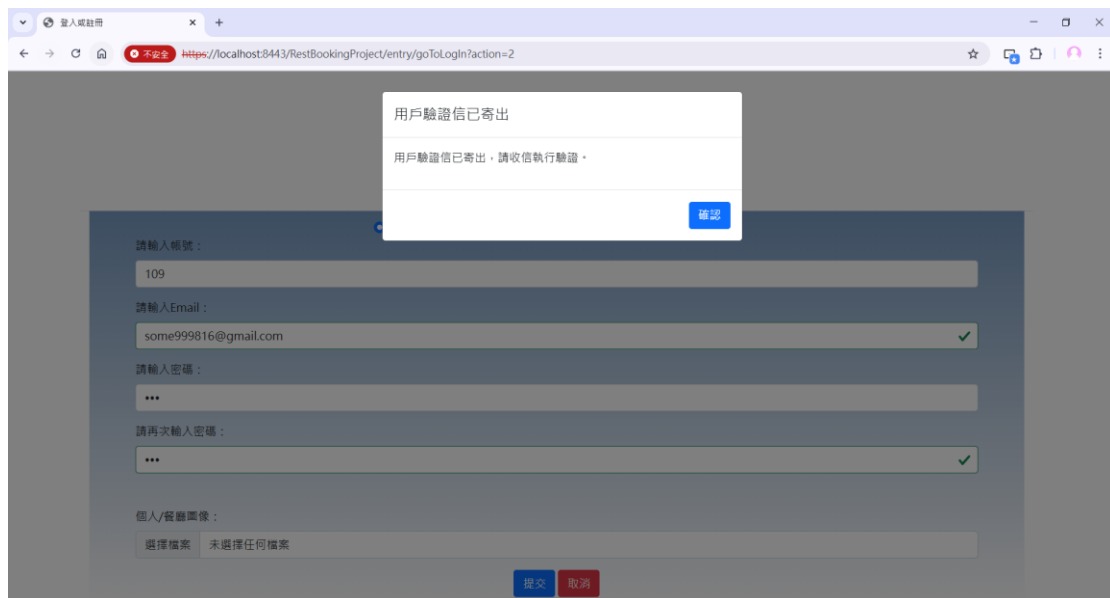
取得明文版驗證碼，那就能寄信了。

```
if(token!=null) {
    emailService.sendVerificationMail(registerDTO.getEmail(), token);
}
```

回傳訊息給回呼函數，讓他可以彈出 modal 來告知用戶收信。

```
return "用戶驗證信已寄出";
```

畫面：



email :

用戶驗證  收件匣 

test@rcyang.bid

寄給我 

https://localhost:8443/RestBookingProject/entry/verify?token=F-NkTvwVwHOrzt8tUMek_xHHEyjd30BRDnz_zv5vxl

下午6:27 (1分鐘前)

3.3 驗證用戶：

點擊連結後就進入 Controller：

3.3.1 Controller：

EntryController：

```
@GetMapping("/verify")
    public String verify(@RequestParam String token, RedirectAttributes ra)
    {
        int result = userService.verify(token);
    }
```

RedirectAttributes redirectAttribute，這個參數是方法參數注入。

3.3.2 Service：

UserService：

將明文版驗證碼轉為密文版，來進行比對，若比對成功則修改用戶狀態、將驗證碼改為已使用。

```
@Transactional("jpaTxManager")
    public int verify(String rawToken) {
        Optional<VerificationToken> result =
            verificationRepository.fetchByToken(token);

        if(result.isPresent()) {
            VerificationToken vt = result.get();
            vt.setUsed(true);
            verificationRepository.save(vt);

            User user = result.get().getUser();
            //user.setEnabled(true);

            return userDao.updateUser(user);
        }
    }
```

fetchByToken 方法是自訂的 Spring Data JPA 方法：

```
@Query("select vt from VerificationToken vt where vt.token = :token")
    Optional<VerificationToken> fetchByToken(@Param("token") String token);
```

將驗證碼改為已使用：

```
verificationRepository.save(vt);
```

Spring Data JPA 的 save，會根據主鍵是否存在，判斷要做 insert 還是 update。

修改用戶狀態：

```
userDao.updateUser(user);
```

3.3.3 回到 Controller：

回傳訊息給登入頁面，以呈顯含訊息的 alert 視窗：

```
if(result!=0) {  
    ra.addFlashAttribute("verificationPass", "用戶已通過驗證");  
    return "redirect:/entry/goToLogIn";  
}  
return null;
```

addFlashAttribute(key, val) 將資料先暫存(通常在 Session)，再放入下一請求的 request，才能用 el 取出資料；addAttribute(key, val)會將資訊直接曝露在 querystring，故較不適合。

在 Spring MVC，"redirect:/entry/goToLogIn" 路徑直接會被加上 ContextPath。

3.3.4 回到頁面：

regAndLogin5.jsp：

```
<c:if test="${not empty verificationPass}">  
    <div class="alert alert-success position-relative">  
        ${verificationPass}  
        <button type="button" class="btn-close position-absolute end-0  
me-2" data-bs-dismiss="alert" aria-label="Close"></button>  
    </div>  
</c:if>
```

position-absolute 是根據上層元素來做絕對定位？根據上層有 position: relative 的元素。

畫面：



3.4 遇到的錯誤：

3.4.1 java.lang.IllegalStateException: Already value
[org.springframework.jdbc.datasource.ConnectionHolder@4a31d267] for key [HikariDataSource (HikariPool-1)]
bound to thread

添加@Transactional("jpaTxManager")

3.4.2 org.springframework.beans.factory.NoSuchBeanDefinitionException: No bean named 'transactionManager'
available: No matching TransactionManager bean found
for qualifier 'transactionManager' - neither qualifier
match nor bean name match!

添加

@EnableJpaRepositories(transactionManagerRef = "jpaTxManager")

這樣就能取代 bean 註冊檔語法：

```
<tx:annotation-driven-  
manager="transactionManager"></tx:annotation-driven>
```

到 Spring Boot 啟動類

3.4.3 java.sql.SQLException: Field 'createdAt' doesn't have a default value

資料庫欄位名改成 `created_at`

3.4.4 出現 Failed to authenticate since password does not match stored value

因為註冊密碼，`encode` 了兩次。

4 登入功能：

首先會用 <http://localhost:8080/RestBookingProject/index.jsp>

來登入首頁，接著要能自動跳轉到

<https://localhost:8443/RestBookingProject/index.jsp>

4.1 如何設置 http 請求，自動跳轉到 https：

`TestSpringSecurity5xConfig.java`：

透過 Spring Security，先設置 http 8080 請求專用的通道：

4.1.1 設置 http 8080 請求專用的通道：

@Bean

```
public WebServerFactoryCustomizer<TomcatServletWebServerFactory>
customizer() {
    return factory ->
factory.addAdditionalTomcatConnectors(createHttpConnector());
}

private Connector createHttpConnector() {
    Connector connector = new
Connector(TomcatServletWebServerFactory.DEFAULT_PROTOCOL);
    connector.setScheme("http");
    connector.setPort(8080);
    connector.setSecure(false);

    return connector;
}
```

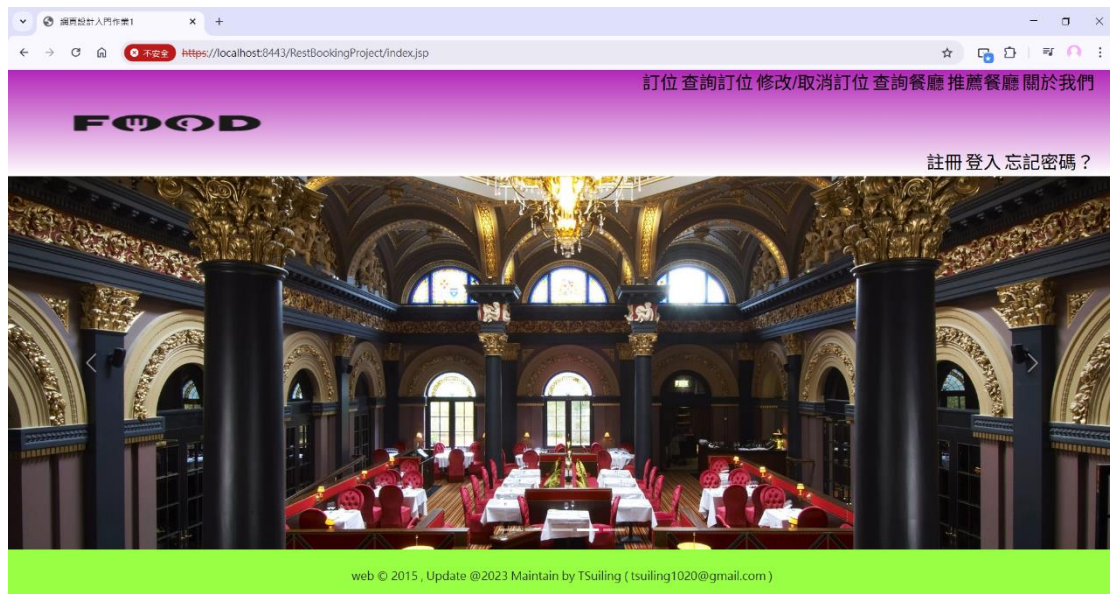
```
}
```

4.1.2 處理來自 http 通道的請求：

來到專門處理來自 http 通道的請求，其先將 http url 改成 https url，包在 response 內，並回覆要求瀏覽器重導的 response 給瀏覽器，讓瀏覽器用 https 的 url，進行重導。：

```
.requiresChannel(channel -> {  
    channel.anyRequest().requiresSecure();  
}).portMapper((portMapper) ->  
portMapper  
.http(8080).mapTo(8443)  
)
```

重導結果：



4.2 驗證與授權：

4.2.1 設置驗證用戶工具：

```
.authenticationProvider(customProvider)
```

AuthenticationProvider 讓 Spring Security 能根據用戶清單，檢查嘗試登入者是否為用戶。

有了 AuthenticationProvider，外加透過 Spring Security 驗證用戶登入成功，就可以在 Spring Security tag 使用 principal。

authenticationProvider(customProvider)，這段設置驗證方式，但不執行驗證；真正的驗證->不用在 securityFilterChain 內定義。

4.2.2 授權：

```
.authorizeRequests(authorizeRequests -> authorizeRequests
    .antMatchers("/entry/goToLogIn",
"/entry/checkLogin", "/index.jsp")
    .permitAll().anyRequest().authenticated())
```

設置請求的授權方式：分別設置可直接授權的請求，跟必須經過驗證才可授權的請求。

4.2.3 驗證：

這裡只設定表單驗證：

```
.formLogin(form -> form
    .loginPage("/entry/goToLogIn").usernameParameter("account")
    .passwordParameter("password")
    .loginProcessingUrl("/entry/checkLogin")
    .defaultSuccessUrl("/entry/login"))
```

loginProcessingUrl 表示該 url 對應到 Spring Security 自帶的驗證功能，完全不需自己另外寫方法來驗證，故該 url 無對應方法。

defaultSuccessUrl 方法，表示驗證成功後，重導要用的 url；這個 url 就可對應到某方法，以便做 forward。這表示若要 forward 到某個 jsp 頁面，就必須用表單提交。

4.2.3.1 forward 的問題：

Controller 方法明明有跑到 return "xxx"，且 viewResolver 正確設置，卻無法呈顯 xxx.jsp，瀏覽器頁面看起來是停留在同一頁，且完全無錯誤訊息，連 eclipse console 都無錯誤訊息？

這是因為，用 fetch/XHR 送出請求。如果用 JQuery ajax，也是一樣。

4.2.3.2 為何跳出 Spring Security 預設登入頁面：

UserDetailsService 的 loadUserByUsername 方法丟了例外。

帳戶過期、被鎖住(isAccountNonExpired、isAccountNonLocked 都是 true)。
都會導致重導至 Spring Security 預設登入頁面。

4.3 開啟 CSRF 功能：

TestSpringSecurity5xConfig.java：

要讓使用者登入後，Spring Security 給這位使用者 csrf token。

```
.csrf(csrf -> csrf  
    .csrfTokenRepository(new HttpSessionCsrfTokenRepository()))
```

4.3.1 HttpSessionCsrfTokenRepository，是將 SESSION 跟

CSRF TOKEN 綁定，為何這樣可以阻止 CSRF 攻擊？

伺服器針對所有請求中，額外加入一個隨機字串（CSRF Token），並傳回給瀏覽器端，瀏覽器才能給請求添加這個 CSRF Token。這樣的話，即使有心人士企圖偽造請求，也會因為不知道伺服器給了什麼 CSRF Token 而無法偽造，也就阻止了 CSRF 攻擊。

4.3.2 不過，有心人士難道不能於惡意程式內，添加

```
document.querySelector('meta[name="_csrf"]').content;
```

來獲取 CSRF Token？

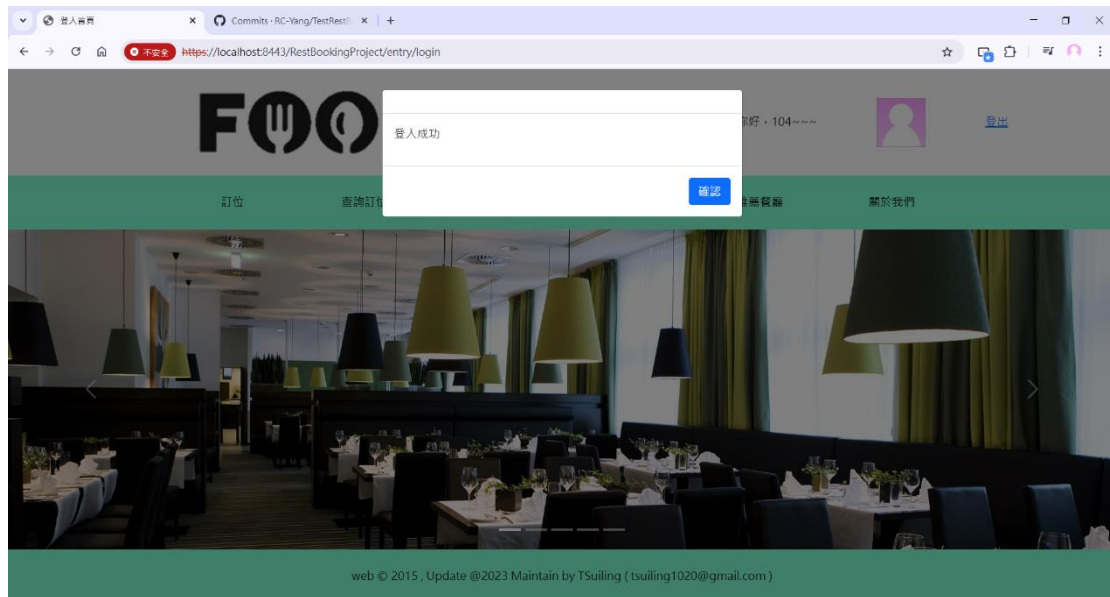
不能，因為有心人士偽造的網頁，網域是別的，別網域的網站無法直接取得 CSRF Token。

4.3.3 只能用 HttpSessionCsrfTokenRepository 來綁定

CSRF Token？

伺服器的 Session 跟瀏覽器的 cookie 內的 jsessionid，都是用戶在伺服器端的唯一識別碼，所以都可以用來綁定 CSRF Token。

4.4 登入成功頁面：



5 登出功能：

透過 Spring Boot+Spring Security，來設置登出+CSRF 功能。

5.1 設置 session 到期時間：

於 application.properties 內設置：

```
server.servlet.session.timeout=2m
```

5.2 登出功能：

5.2.1 後端部分：

TestSpringSecurity5xConfig.java：

```
.logout(logout -> logout
    .logoutRequestMatcher(new OrRequestMatcher(
        new AntPathRequestMatcher("/session/logout", "POST"),
        new AntPathRequestMatcher("/entry/logout", "GET")
    ))
    .logoutSuccessUrl("/index.jsp")
    .invalidateHttpSession(true)
```

```
.deleteCookies("JSESSIONID"))
```

`logoutSuccessUrl` 方法內的 `url` 參數，會跟重導通知，被伺服器包裝進 `response`，這樣就會重導至該頁面。

這樣即可，不需另外寫 `Controller` 方法。

5.2.2 前端部分：

5.2.2.1 以超連結來傳送登出資訊：

`loginSuccessForUser.jsp`：

```
<a href="<%=request.getContextPath() %>/entry/Logout">登出</a>
```

5.2.2.2 以表單來傳送登出資訊：

`remindSessionTimeoutModal.jsp`：

```
<form id="LogoutForm" action="<%= request.getContextPath() %>/session/logout" method="POST">
  <input type="hidden" name="_csrf" value="${_csrf.token}" />
</form>
```

```
const form = document.querySelector("#logoutForm");
form.submit();
```

5.2.2.3 以表單+fetch API 來傳送登出資訊：

```
const form = document.querySelector("#logoutForm");
const formData = new FormData(form);

fetch(form.action, {
  method: form.method,
  body: formData,
  credentials: "same-origin" // 夾帶 JSESSIONID cookie
}).then(res => { if (res.redirected) { window.location.href = res.url; } });
```

`fetch` 方法執行後，等待，直到後端回傳一個物件，比方說 `response` 物件，這時 `Promise` 物件的狀態，會變成 `fulfilled`。

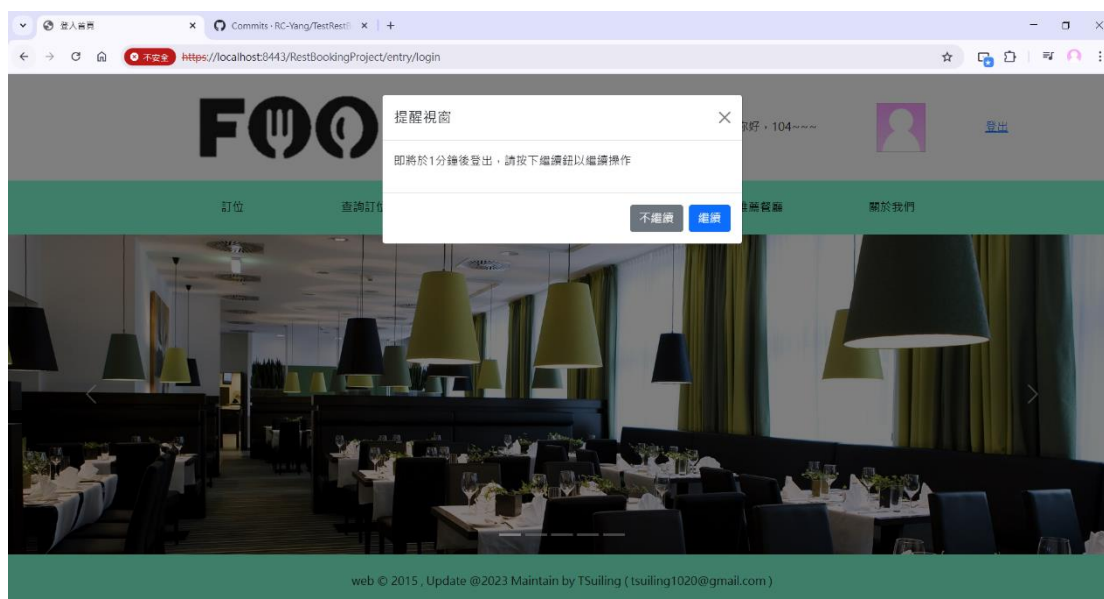
res 就是 JavaScript 的 response 物件。

5.2.2.4 以 fetch API 來傳送登出資訊：

```
const token = document.querySelector('meta[name="_csrf"]').content;
const header = document.querySelector('meta[name="_csrf_header"]').content;
fetch('<%= request.getContextPath() %>/session/logout', {
  method: 'POST',
  credentials: 'same-origin',
  headers: { [header]: token }
}).then(res => { if (res.redirected) { window.location.href = res.url; } });
```

6 按下特定按鈕後，延長 session 存活時間以繼續維持登入：

設置 BootStrap modal，再設置按下按鈕後，發送一個請求到後端即可：



6.1 後端部分：

SessionController.java：

```
@GetMapping("/keepAlive")
@ResponseBody//將已被 Spring Boot 轉成 json 格式字串的物件，寫入 response body
public Map<String,Object> keepAlive() {
```

```
        return Map.of("ifContinue", true);
    }
}
```

6.2 前端部分：

remindSessionTimeoutModal.jsp：

```
document.getElementById("continue").addEventListener("click",function(){
    fetch("<%= request.getContextPath() %>/session/keepAlive")
        .then((res)=>res.json())
        .then((map)=>{
            if(map.ifContinue){
                remindSessionTimeout();
                remindSessionTimeoutModal.hide();
            }
        });
});
```

`then()`，一樣是產生回傳物件後，該物件被包裹在 `promise` 物件裡，且 `promise` 物件狀態改為 `fulfilled`。

`res` 就是 JavaScript 的 `response` 物件。

在 Spring Boot，後端回傳的物件，被額外處理成 JSON 格式的內容；若還添加了 `@ResponseBody`，則該 JSON 格式的內容還會被寫入 `response body`。要解析成對應的 JavaScript 物件，就要用 `res.json()`。

當 `res.json()`處理完成，`then` 函數就將其包裹在 `promise` 物件裡，這時 `promise` 物件狀態改為 `fulfilled`；接著便 `return promise` 物件，給下一個 `then` 函數。

`then((res)=>res.json())`，等價於 `then((res)=>{return res.json();})`。

7 重設密碼功能：

要透過忘記密碼頁面，輸入 `email`，於表單按下確認後，彈出確認視窗，按下確認後，寄發修改密碼連結信給該 `email`，透過該連結進入修改密碼頁面，進而修改密碼。

7.1 前端：

reqAndLogin5.jsp

7.1.1 彈出重設密碼確認視窗：

```
function submitEmailForm(){
    if(checkEmailForm()==true){

        var sendMailConfirmModal =
document.getElementById("sendMailConfirm");
        sendMailConfirmModal = new bootstrap.Modal(sendMailConfirmModal);

        sendMailConfirmModal.show();

        //該區塊要監聽該 modal 是否被 click，有的話就關閉該 modal、寄發密碼重設連結信給表單內填寫的 email，寄發成功後彈出另一 modal，表示已成功寄出，監聽該 modal 是否被 click，有的話就返回首頁
    }
}
checkValidity，是瀏覽器的原生 API；直接檢查所有 input 欄位值是否合法。
```

7.1.2 寄發修改密碼連結信給該 email：

那麼，在函數內監聽事件，會因為函數結束而跟著無法繼續監聽？
在函數內監聽事件，不會隨著函數執行結束而結束。這其實就是閉包。

7.1.2.1 閉包：

閉包是一個 block+外部環境參照，即使外部環境被移除仍能持續存在，內部參照會隨著外部修改而改變。

最常見的閉包就是函數內的函數，比方說 `addEventListener`、`setTimeout` 內的函數參數。

7.1.2.2 事件冒泡：

內層、外層 DOM 元素若監聽相同事件，然後內層元素觸發該事件，如果外層也跟著觸發事件，就是事件冒泡；為了避免這樣，就需要在內層函數添加

`e.stopPropagation();`

如果外層 DOM 元素沒有監聽相同事件，就可不加 `e.stopPropagation();`

```
document.querySelector('#sendMailConfirm button.btn-
primary').addEventListener("click",function(){

    //$('#sendMailConfirm').modal('hide');
    sendMailConfirmModal.hide();

    var forgetPasswordForm = document.getElementById("forgetPassword");
    forgetPasswordFormData = new FormData(forgetPasswordForm);

    fetch("<%=request.getContextPath() %>/entry/sendUpdatePasswordMail",
        {method:"post",body: forgetPasswordFormData})
        .then(res=>res.text())
        .then((text)=>{
            if(text=="密碼重設信傳送成功"){
                var sendMailFinish =
document.getElementById("sendMailFinish");
                sendMailFinishModal = new bootstrap.Modal(sendMailFinish);

                //$('#sendMailFinish').modal('show');
                sendMailFinishModal.show();

                document.querySelector('#sendMailFinish button.btn-
primary').addEventListener("click",function(){
                    //sendMailFinishModal.hide();

                    setTimeout(()=>location.href="https://localhost:8443/RestBookingProject
/index.jsp",300);

                });
            }
        });
});
```



```
});
```

Spring Boot 內的 jQuery ajax 回呼函數，若是後端回傳字串，則回呼函數的參數，值即為該字串。

`new bootstrap.Modal`，參數可以是 DOM 元素跟 CSS 選擇器？只能是前者。

上面的表單以 `FormData` 物件格式傳輸，那可以用

```
$('#updatePasswordForm').serialize() ?
```

`$('#updatePasswordForm').serialize()`，這樣是將這個 jQuery 物件內的值序列化，成為可加在 url 後方的 `querystring` 或被寫入 `HttpRequest`。

`$('#updatePasswordForm').serialize()`和 `FormData` 物件差異，就是兩者都能被寫入 `HttpRequest`，但後者還能另外寫入多媒體訊息。

以上的語法，可以寫成 `onclick="showConfirmModal();" ?`

這種語法，無法設定 `nonce`，故會有 XSS 的可能。

7.1.3 防止 XSS 的方式：

透過 `filter`，已在 `HttpResponse` header 的 `script-src` 裡，放了 `'nonce-xxxx'`；這是必須的，因為這樣瀏覽器才能拿 `response` 裡面的 `nonce` 來跟 `script` tag 內的 `nonce` 來比對是否一致。

這樣的話，於 `response.setHeader` 設置 `unsafe-inline`，`unsafe-inline` 會無效；最好 `'unsafe-inline'` 跟 `'unsafe-eval'` 要移除。

7.1.4 Bootstrap modal 淡入淡出效果不見的問題：

```
bootstrap modal 有設置 fade class，但 sendMailFinishModal.show();  
sendMailFinishModal.hide();
```

卻沒有淡入淡出效果？

因為程式某處將 `fade class` 移除了。

7.2 後端：

7.2.1 application.properties：

於 application.properties 設定 Spring Mail：

```
spring.mail.host=email-smtp.ap-southeast-1.amazonaws.com
spring.mail.port=587
spring.mail.username=${mail.username}
spring.mail.password=${mail.password}
spring.mail.properties.mail.smtp.auth=true
spring.mail.properties.mail.smtp.starttls.enable=true
```

這裡的寄信伺服器是 AWS SES，所以 username = 連到 AWS SES 且有 SMTP 憑證的帳號。

上面的帳密不能被提交至遠端，故必須寫在另一個不會被提交的檔案內：

檔案要置於 src/main/resources。檔案名叫 secrets/mail.properties

然後在 application.properties，引用該檔案：

```
spring.config.import=optional:classpath:secrets/mail.properties
```

7.2.2 設置 mail.properties 不會被提交：

7.2.2.1 .gitignore 檔：

於.gitignore 檔，添加/RestBookingProject/src/main/resources/secrets/

最前面多一個斜槓，是因為要表示從 repository 開始的路徑；結尾有/，表示 secrets 資料夾本身直接被排除在 git 管理。

7.2.2.2 git status：

存檔後，下 git status 檢查是否成功，若出現

Untracked files: (use "git add <file>..." to include in what will be committed)

RestBookingProject/src/main/resources/secrets/

表示剛剛在.gitignore 檔，寫的路徑有誤。

7.2.3 Controller :

```
@RequestMapping("/sendUpdatePasswordMail")
@ResponseBody
public String sendUpdatePasswordMail(HttpServletRequest req) {
    String email = req.getParameter("email");

    emailService.sendMail(email);
    return "密碼重設信傳送成功";
}
```

7.2.4 Service :

```
@Service
public class EmailService {

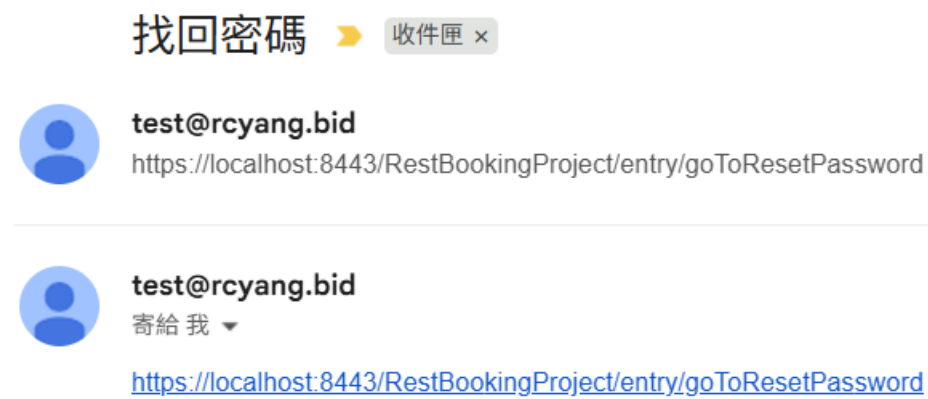
    @Autowired
    private JavaMailSender javaMailSender;

    public boolean sendMail(String email) {
        SimpleMailMessage message = new SimpleMailMessage();
        message.setFrom("test@rcyang.bid");
        message.setTo(email);
        message.setSubject("找回密碼");

        message.setText("https://localhost:8443/RestBookingProject/entry/goToResetPassword");

        try {
            javaMailSender.send(message);
            return true;
        } catch (Exception e) {
            System.out.println(e);
            return false;
        }
    }
}
```

寄信結果：



7.3 回到前端：

passwordReset.jsp

```
function resetPasswordWithAjax(){
    $.post({
        url: '/RestBookingProject/entry/updatePassword',
        data: $('#updatePasswordForm').serialize(),
        success: (data) => {
            if (data == 'update password success') {
                $('#updatePasswordSuccess').modal('show');
            }
        },
        error: () => {
        }
    });
}
```

8 查詢餐廳功能：

8.1 設置查詢餐廳頁面：

queryRest4.jsp

使用 bootstrap 手風琴 ui+bootstrapVue 套件來呈顯城市與行政區的 checkbox，且切換城市的 checkbox 時可以動態改變成對應的行政區 checkbox，且原本選擇的行政區 checkbox 不會不見。

城市的 checkbox 區，要有呈顯的 option 本身、被選中的 value 兩種值，這兩種值來自 Vue 的 data 參數；計畫要從後端取出城市的 id、名稱，取出的內容直接寫入這個參數，這樣 checkbox 區就能自動呈顯 checkbox 跟城市名稱。

行政區的 checkbox 區，一樣要有呈顯的 option 本身、被選中的 value 兩種值，這兩種值也來自 Vue 的 data 參數。

另外還要能透過 bootstrapVue 標籤，呈顯已被選中的資料，資料一樣來自 Vue 的 data 參數。

這樣 Vue 的 data 參數，內部總共需要五個參數；又因為這些參數，不管是城市還是行政區還是呈顯的標籤都是多個，故參數必須設為陣列：

```
data(){  
    return{  
        cityId:'',  
        cityOptions:[],  
        districtId:[],  
        districtOptions:[],  
        allDistricts:[]  
    }  
}
```

cityId、districtId 分別是要被綁定到城市、行政區 checkbox 的 value；cityOptions、districtOptions 分別是要被綁定到城市、行政區 checkbox 的 option 本身。

其實這樣就是 return javascript 物件。

8.1.1 頁面載入：

城市、行政區需要有預設呈顯的資料，故頁面載入時就必須執行 fetch()，從後端取出相關資料，寫入 Vue 參數，讓 Vue 可以即時渲染這些資料：

```
mounted(){  
    fetch("<%=request.getContextPath()%>/form/queryAllCity")  
    .then(res=>res.json()).then(data=>this.cityOptions=data);
```

```

    fetch("<%=request.getContextPath()%>/form/queryDistrictForRest?country=
    臺北市")
        .then(res=>res.json()).then(data=>this.districtOptions=data);
    }

```

bootstrapVue 套件的 option 是接收 js array，而後端 List 傳回前端，經過 json() 後，就是 JS Array。

頁面載入時，另外還需要先將全部的行政區資料都準備好，這樣用戶按下行政區時，就可及時檢查哪個行政區被選中，需要用標籤來呈顯：

```

fetch("<%=request.getContextPath()%>/form/queryAllDistricts")
    .then(res=>res.json()).then(data=>this.allDistricts=data)

```

8.1.2 用戶每選中一個 checkbox 的相應動作：

用戶每選中一個 checkbox，就要同步運算有哪些標籤要呈顯：

```

computed:{
    selectedOptions(){
        var arr;
        arr=this.allDistricts.filter(district=>this.districtId.includes(district.districtId));
        return arr;
    }
}

```

上面的 this.districtId，是 Vue 參數且被綁定到 checkbox 的 value，故可直接拿 this.districtId 跟 this.allDistricts 的 id 來比。

8.1.3 用戶每切換一個城市的相應動作：

用戶每切換一個城市，就要同步執行方法來切換城市的行政區：

```

methods:{
    loadDistricts(){

        fetch("<%=request.getContextPath()%>/form/queryDistrict?cityId="+this.cityId)
            .then(res=>res.json()).then(data=>this.districtOptions=data)
        }
    }
}

```

checkbox 那邊需要用 change 事件來呼叫方法：

@change="loadDistricts"

8.1.4 用 bootstrapVue 套件做的 checkbox 群組：

```
<b-form-group>
<b-form-checkbox-group v-model="cityId"
                        :options="cityOptions"
                        value-field="id"
                        text-field="countryName"
                        @change="loadDistricts">

</b-form-checkbox-group>
<hr>
<b-form-checkbox-group v-model="districtId"
                        :options="districtOptions"
                        value-field="districtId"
                        text-field="districtName"
                        name="districtId">

</b-form-checkbox-group>
</b-form-group>
```

v-model 表示 checkbox 值要跟哪個 Vue 參數綁定 (v-model 用於將外部值綁定到 data 屬性值)。

:options 表示 checkbox option 群組要跟哪個 Vue 參數綁定。

value-field、**text-field** 是用於對應後端資料的屬性名稱。

name 是用於 checkbox 值對應的 name 名稱。

另外，為了使頁面 ui 整齊，故還需添加 **style="column-count:3;**

8.1.5 使用表單傳送資料：

```
<b-form
      action="<%=request.getContextPath()%>/rest/queryRests"
      method="post">
<b-button type="submit" class="btn btn-success my-2">查詢</b-button>
</b-form>
```

另外為使表單 ui 整齊，故添加

class="d-flex flex-column justify-content-center align-items-center vh-100"

8.1.6 查詢餐廳頁面：

依地區選擇餐廳

☐ 基隆市	☐ 新竹市	☒ 臺北市
☐ 宜蘭縣	☐ 新竹縣	☐ 連江縣
☐ 新北市	☐ 桃園市	☐ 釣魚台

☒ 中正區	☐ 大安區	☐ 北投區
☐ 大同區	☐ 萬華區	☐ 內湖區
☐ 中山區	☒ 信義區	☐ 南港區
☐ 松山區	☐ 士林區	☐ 文山區

依風格選擇餐廳

8.2 後端：

8.2.1 DAO：

```
@Query(value="SELECT d.* FROM district d join country c"
        + " on d.country_name=c.country_name WHERE c.id = :cityId"
        ,nativeQuery=true)
List<District> findDistrictsByCity(@Param("cityId")Integer cityId);

public String getDistrictNameById(String districtId) {
    String sql="select district_name from web.district where
district_id=?";

    String queryCondition = districtId;

    RowMapper<String> mapper = new RowMapper<>() {
        public String mapRow(ResultSet rs, int rowNum) throws
SQLException {
            return rs.getString("district_name");
        }
    };
};
```



```

        return jdbcTemplate.queryForObject(sql, mapper, queryCondition);
    }

```

8.2.2 Controller :

```

@RequestMapping(value="/queryRests")
public String queryRests(Model model, @RequestParam String[] districtId)
{

    List<Restaurant> allQueryRest = new ArrayList<>();

    List<String> districtNames=new ArrayList<>();
    for(String id:districtId) {
        String districtName = restDao.getDistrictNameById(id);
        districtNames.add(districtName);

        List<Restaurant> rests =
restDao.getRestsByDistrictJoinImage(districtName);
        for(Restaurant rest:rests) {
            allQueryRest.add(rest);
        }
    }

    model.addAttribute("checkedDistrictList", districtNames);
    model.addAttribute("rests", allQueryRest);

    return "queryRestResult2";
}

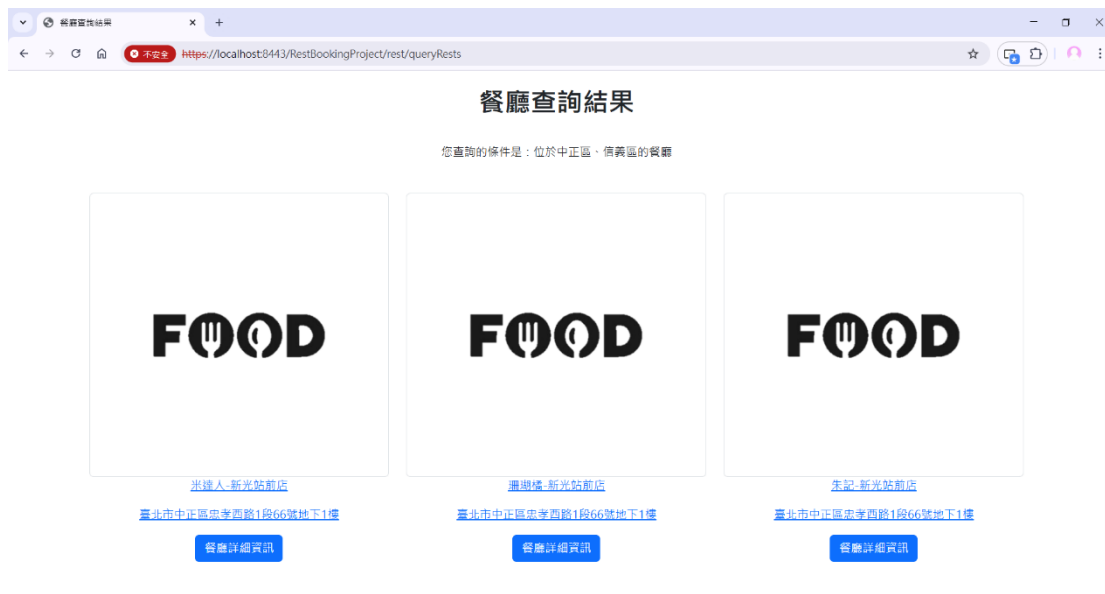
```

@RequestParam(value = "districtId") String[] districtId

可以簡寫為

@RequestParam String[] districtId

8.2.3 查詢結果：



8.3 錯誤排查：

8.3.1 查詢頁面的手風琴 ui，一開始無法完全置中，於上層元素使用 d-flex 也一樣：

添加 `vh-100`。

8.3.2 在執行查詢行政區的 Spring Data JPA 部分時，出現找不到 id 的錯誤：

因為 SQL 必須完全將 VO 的所有屬性值都查出，否則會無法塞值給 VO 來回傳。

8.3.3 瀏覽器開發者模式下的程式一直沒有更新：

關掉開發者模式重開、重新執行程式即可。